



Access S3 from a VPC

JJ

jjoseph1608@outlook.com

```
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls s3://nextwork-vpc-project-jordan-joseph
2026-06-05 15:03:42      2431554 NextWork - Denzel is awesome.png
2026-06-05 15:03:42      2399812 NextWork - Lelo is awesome.png
[ec2-user@ip-10-0-13-76 ~]$ sudo touch /tmp/test.txt
[ec2-user@ip-10-0-13-76 ~]$ ls
[ec2-user@ip-10-0-13-76 ~]$ pwd
/home/ec2-user
[ec2-user@ip-10-0-13-76 ~]$ ls -a
.  ..  .aws  .bash_history  .bash_logout  .bash_profile  .bashrc  .ssh
[ec2-user@ip-10-0-13-76 ~]$ aws s3 cp /tmp/test.txt s3://nextwork-vpc-project-jordan-joseph
upload: ../../tmp/test.txt to s3://nextwork-vpc-project-jordan-joseph/test.txt
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls s3://nextwork-vpc-project-jordan-joseph
2026-06-05 15:03:42      2431554 NextWork - Denzel is awesome.png
2026-06-05 15:03:42      2399812 NextWork - Lelo is awesome.png
2026-06-05 15:08:46           0 test.txt
[ec2-user@ip-10-0-13-76 ~]$
```



Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is a service that lets you create isolated private networks within AWS, and it is useful because it gives you full control over your network environment including IP ranges, subnets, route tables, and security settings, keeping your cloud resources separate and secure

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to create a private network and launch an EC2 instance into it, giving me a secure environment to configure AWS CLI access and interact with Amazon S3 from the command line

One thing I didn't expect in this project was...

One thing I didn't expect in this project was how straightforward it was to interact with S3 from an EC2 instance once the access keys were configured. A few simple CLI commands were all it took to create a file and upload it to a bucket, which made the connection between EC2 and S3 feel very tangible

This project took me...

This project took me around 30 minutes to complete, including setting up the VPC, launching the EC2 instance, configuring the access keys, creating the S3 bucket, and testing the connection through the AWS CLI



In the first part of my project...

Step 1 - Architecture set up

In this step, I will create a VPC and launch an EC2 instance into it, because this gives me a running server inside a private network that I can later configure to communicate with Amazon S3 using access keys

Step 2 - Connect to my EC2 instance

In this step, I will connect to my EC2 instance using EC2 Instance Connect, because I need to get inside the instance and access its command line to run the AWS commands that will interact with Amazon S3

Step 3 - Set up access keys

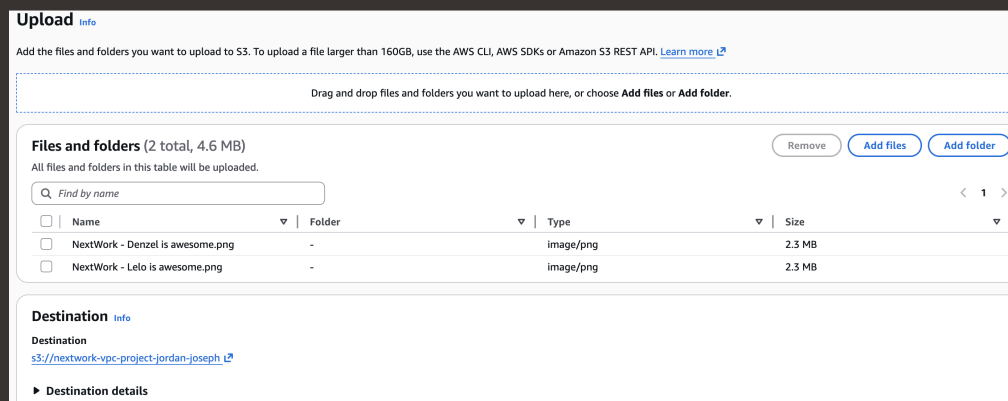
In this step, I will generate access keys and configure them on my EC2 instance, because this gives the instance the credentials it needs to authenticate with AWS and start interacting with Amazon S3 through the command line



Architecture set up

I started my project by launching a VPC with a public subnet, an internet gateway, and a route table, and then launched an EC2 instance into it so I have a server ready to connect to Amazon S3

I also set up my S3 bucket with a unique name and selected a region, giving me a storage container in Amazon S3 that my EC2 instance can interact with through the AWS CLI



Running CLI commands

AWS CLI is a command line tool that lets you interact with AWS services directly from a terminal by running text commands instead of clicking through the console. I have access to AWS CLI because it comes pre-installed on Amazon Linux EC2 instances by default

The first command I ran was `aws s3 ls`. This command is used to list all of the S3 buckets in your AWS account, and I ran it to test whether my EC2 instance had the necessary permissions to communicate with Amazon S3

The second command I ran was `aws configure`. This command is used to set up the AWS CLI with the credentials and region needed to authenticate requests, so that my EC2 instance has the permission to interact with my AWS account and its services

```

#
--\##### Amazon Linux 2023
--\#####\
--\#####|
--\##/ https://aws.amazon.com/linux/amazon-linux-2023
--\v~'--->
--
--\_
--\_/_m/
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls

Unable to locate credentials. You can configure credentials by running "aws login".
[ec2-user@ip-10-0-13-76 ~]$ aws configure
AWS Access Key ID [None]:

```



Access keys

Credentials

To set up my EC2 instance to interact with my AWS environment, I configured my access key ID, secret access key, and default region using the `aws configure` command, which stores these credentials on the instance so that every AWS CLI command I run is authenticated against my AWS account

Access keys are a pair of credentials made up of an access key ID and a secret access key that are used to authenticate programmatic requests to AWS, allowing tools like the AWS CLI to interact with your AWS account and its services on your behalf

Secret access keys are the private part of an access key pair that acts like a password, used alongside the access key ID to authenticate requests to AWS. They should be kept secure and never shared, as anyone with access to them can make requests to your AWS account on your behalf

Best practice

Although I'm using access keys in this project, a best practice alternative is to use IAM roles attached directly to the EC2 instance, because this avoids storing long term credentials on the instance and instead lets AWS handle authentication automatically and securely



In the second part of my project...

Step 4 - Set up an S3 bucket

In this step, I will launch an S3 bucket, because I need a storage container in Amazon S3 that my EC2 instance can interact with to test that the access keys and CLI configuration are working correctly

Step 5 - Connecting to my S3 bucket

In this step, I will use the AWS CLI on my EC2 instance to interact with my S3 bucket, because this is how I confirm that the access keys are working correctly and my instance has the permissions it needs to read and write objects in Amazon S3



Connecting to my S3 bucket

The first command I ran was `aws s3 ls`. This command is used to list all of the S3 buckets in your AWS account, and I ran it to test whether my EC2 instance had the necessary permissions to communicate with Amazon S3

When I ran the `aws s3 ls` command again, the terminal responded with a list of my S3 buckets. This indicated that my EC2 instance was now successfully authenticated and able to communicate with Amazon S3 using the access keys I had configured

```
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls
2026-06-05 14:19:35 nextwork-vpc-project-jordan-joseph
[ec2-user@ip-10-0-13-76 ~]$
```



Connecting to my S3 bucket

Another CLI command I ran was `aws s3 ls s3://nextwork-vpc-project-jordan-joseph` which returned a list of the objects stored inside that specific S3 bucket, confirming that my EC2 instance could successfully access and read the contents of my bucket

```
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls s3://nextwork-vpc-project-jordan-joseph
2026-06-05 15:03:42      2431554 NextWork - Denzel is awesome.png
2026-06-05 15:03:42      2399812 NextWork - Lelo is awesome.png
[ec2-user@ip-10-0-13-76 ~]$
```



Uploading objects to S3

To upload a new file to my bucket, I first ran the command `sudo touch /tmp/test.txt`. This command creates a new empty file called `test.txt` in the `tmp` directory on my EC2 instance, giving me something to upload to my S3 bucket

The second command I ran was `aws s3 cp /tmp/test.txt s3://nextwork-vpc-project-jordan-joseph`. This command will copy the `test.txt` file from my EC2 instance and upload it to my S3 bucket, testing that my instance has the correct permissions to write objects to Amazon S3

The third command I ran was `aws s3 ls s3://nextwork-vpc-project-jordan-joseph` which validated that the `test.txt` file had been successfully uploaded to my S3 bucket by showing it appearing in the list of objects stored inside it

```
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls s3://nextwork-vpc-project-jordan-joseph
2026-06-05 15:03:42      2431554 NextWork - Denzel is awesome.png
2026-06-05 15:03:42      2399812 NextWork - Lelo is awesome.png
[ec2-user@ip-10-0-13-76 ~]$ sudo touch /tmp/test.txt
[ec2-user@ip-10-0-13-76 ~]$ ls
[ec2-user@ip-10-0-13-76 ~]$ pwd
/home/ec2-user
[ec2-user@ip-10-0-13-76 ~]$ ls -a
.  ..  .aws  .bash_history  .bash_logout  .bash_profile  .bashrc  .ssh
[ec2-user@ip-10-0-13-76 ~]$ aws s3 cp /tmp/test.txt s3://nextwork-vpc-project-jordan-joseph
upload: ../../tmp/test.txt to s3://nextwork-vpc-project-jordan-joseph/test.txt
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls s3://nextwork-vpc-project-jordan-joseph
2026-06-05 15:03:42      2431554 NextWork - Denzel is awesome.png
2026-06-05 15:03:42      2399812 NextWork - Lelo is awesome.png
2026-06-05 15:08:46           0 test.txt
[ec2-user@ip-10-0-13-76 ~]$
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

